The built-in functions we've just looked at for interacting with function arguments are certainly very useful, but I'm hoping that you've noticed a problem: the lack of type safety. `testArgs()` has precisely no arguments in its declaration; therefore, we cannot enforce the data type at call time. If was `testArgs()` was implemented to handle only string data, and a somehow an object sneaked in during execution, we'd have problems:

```
function testArgs(): void
{
    foreach (func_get_args() as $str) {
        print $str . ' ';
    }
}

testArgs('I', 'hope', 'this', 'makes', 'sense', new stdClass());
```

```
Output:
I hope this makes sense
Fatal error: Uncaught Error: Object of class stdClass could not be
converted to string
```

Nasty. If only there was a way to enforce type… well there is!

PHP 5.6 introduced the "splat" operator `...` which is demonstrated in Chapter 5, "Arrays". In that chapter, we use the `...` operator to transform an array into individual variables (a process called array unpacking). The splat operator will (when used in a function declaration) also build an array from individual function arguments, with the added bonus that we can add a type declaration at the same time. Function parameters prefixed with the splat operator are called variadic parameters. Let's take the previous example and refactor it slightly to use splat:

```
function testArgs(string ...$in): void
{
    foreach ($in as $str) {
        print $str . ' ';
    }
}

testArgs('I', 'hope', 'this', 'makes', 'sense', new stdClass());
```